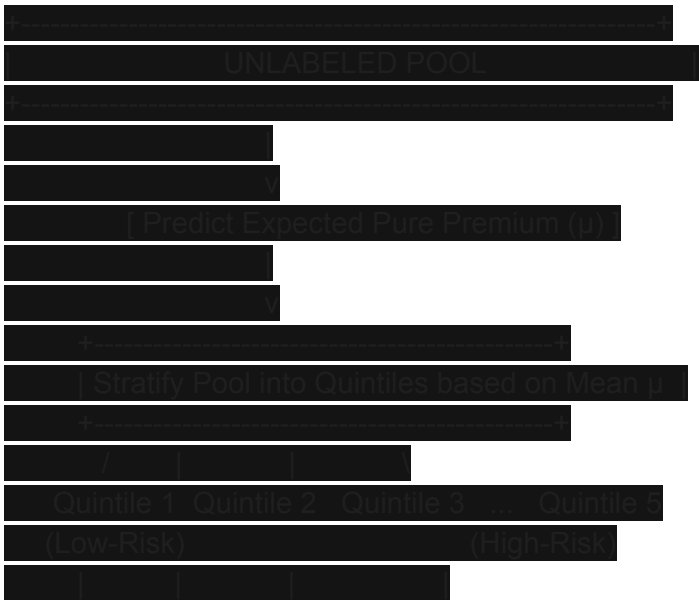


This comprehensive, engineering-first execution blueprint is designed to transition your Active Learning (AL) pipeline from classification to single-model Tweedie pricing. By validating the tree-checkpoint method over the broken pred_contribs approach and defining an explicit mathematical formulation for Zero-Mass Calibration Error (ZMCE), this plan ensures complete code execution readiness.

Formal Problem Statement

- **Objective:** Maximize the labeling ROI of a single Tweedie Gradient Boosted Decision Tree (GBDT) model predicting pure insurance premiums under a strict sequence of 10 active learning feedback iterations.
- **The Technical Bottleneck:** The variance of a Tweedie distribution is bound to its mean by the power law $Var(Y) = \phi \mu^p$. Naive uncertainty sampling selects points with maximum predictive variance, mapping exclusively to severe right-tail claims. This starves the training pool of zero-claim instances, forcing a systemic crash in the model's frequency calibration.
- **The Solution:** Decouple aleatoric noise from epistemic ignorance by tracking the variance of predictions across boosting checkpoints. Combine this with Stratified Expected Deviance Reduction (SEDR) to enforce stable Poisson frequency initialization across low-exposure and high-exposure strata.

2-Day Technical Execution Roadmap





Day 1: Pipeline Adaptation & Calibration Collapse Verification

Goal: Build the single Tweedie data architecture, write the tracking metrics, and empirically capture the "calibration collapse" failure mode of naive AL.

Step 1.1: Data Ingestion and Target Transformation

- **Task:** Load the French Motor Third-Party Liability (MTPL) dataset (freemtpl2freq and freemtpl2sev via OpenML or sklego.datasets). Create the continuous target `pure_premium` defined as:

$$\text{pure_premium} = \frac{\text{ClaimAmount}}{\text{Exposure}} \cdot \text{Exposure}$$
- **Validation Check:** Confirm that over **90%** of rows sit strictly at 0.0, while the remaining rows exhibit a continuous, long right tail.

Step 1.2: Build the Evaluation Suite (Tweedie Deviance & ZMCE)

- **Task:** Code two custom validation metrics to score the test holdout set at each active learning loop iteration.
- 1. **Out-of-Sample Tweedie Deviance:**

$$d(y, \mu) = 2 \left(\frac{y^{2-p}}{(1-p)(2-p)} - \frac{y\mu^{1-p}}{1-p} + \frac{\mu^{2-p}}{2-p} \right)$$
- 2. **Zero-Mass Calibration Error (ZMCE):** Calculate the absolute difference between the true zero-claim proportion in your holdout set and the model's average implied probability of zero.

$$\text{ZMCE} = \left| \frac{1}{N} \sum_{i=1}^N \mathbb{I}(y_i = 0) - \frac{1}{N} \sum_{i=1}^N \exp(-\hat{\lambda}_i) \right|$$

Where the implicit Poisson frequency parameter $\hat{\lambda}_i$ is derived natively from the Tweedie mean prediction ($\hat{\mu}_i$), dispersion (ϕ), and power

parameter (\$p\$):

$$\hat{\lambda}_i = \frac{\hat{\mu}_i^{2-p}}{\phi(2-p)}$$

Step 1.3: Run the Failure-Mode Baselines (10 Iterations)

- **Task:** Implement the iterative feedback loop using a seed set of 2,000 labeled samples and an unlabeled pool. In each of the 10 iterations, query 100 samples based on two distinct baseline policies:
 - **Baseline 1 (Random Passive):** Uniform random selection from the unlabeled pool.
 - **Baseline 2 (Naive Variance AL):** Extract predictions ($\hat{\mu}_i$) from LightGBM, calculate the analytical distribution variance using $\text{Var}(Y) = \phi \hat{\mu}^p$, and query the top 100 maximum variance instances.
- **Deliverable:** Generate a line plot tracking ZMCE across iterations. Baseline 2 must demonstrate an exploding ZMCE curve compared to Baseline 1—proving the core calibration collapse hypothesis.

Day 2: Implementing Epistemic Uncertainty & SEDR

Goal: Implement the correct checkpointing tree disagreement logic, build the stratified selection loop, and benchmark the complete architecture.

Step 2.1: Implement Checkpoint-Based Tree Disagreement

- **Task:** Write the evaluation utility to bypass SHAP-based `pred_contribs` bugs. Extract true epistemic uncertainty by tracking variance across sequential tree-growth checkpoints within your single LightGBM model.
- **Implementation Logic:**

Python

```
n_trees = model.num_trees()
# Step out across 50 increments of tree configurations
step = max(1, n_trees // 50)

# Extract prediction arrays at separate boosting checkpoints
tree_preds = np.array(
    [model.predict(X_pool, num_iteration=k)
     for k in range(step, n_trees + 1, step)]
)

# Calculate Epistemic Uncertainty per instance
J_epistemic = np.var(tree_preds, axis=0)
```

Step 2.2: Implement the SEDR Acquisition Policy

- **Task:** Combine your tree-disagreement array with a stratification mask to protect frequency calibration.
1. Rank the entire unlabeled pool by current predicted mean $\hat{\mu}_i$ and split it into 5

distinct quintiles.

2. Enforce a explicit labeling quota: assign **30%** of the 100-sample query budget strictly to Quintile 1 (the zero-mass/low-risk boundary). Divide the remaining 70% of the budget proportionally across Quintiles 2–5.
3. Within each separate quintile, rank instances strictly by your checkpoint variance $u_{\text{epistemic}}$ and query the top-performing samples to fill the assigned quotas.

Step 2.3: Run Comparative Benchmarking

- **Task:** Execute the full 10-iteration loop using the two advanced strategies:
 - **Baseline 3 (Pure Tree Disagreement):** Global ranking using $u_{\text{epistemic}}$ without stratification.
 - **Method 4 (Tree Disagreement + SEDR):** Your core proposed framework.

Day 3: Industrial Validation & Metric Compilation

Goal: Compute economic utility metrics, run a synthetic validation sweep, and finalize your core project deliverables.

Step 3.1: Calculate Economic Price Discrimination (Lorenz & Gini)

- **Task:** For your final iteration models across all 4 strategies, sort the test holdout population by their predicted premium on the x-axis, and plot the cumulative actual losses on the y-axis (Lorenz Curve).
- **Deliverable:** Compute the Gini index for each curve. A well-calibrated pricing model will maximize the Gini coefficient, mathematically validating that active learning translates directly into superior premium risk discrimination.

Step 3.2: Controlled Parameter Recovery Test

- **Task:** Write a fast 5-minute script generating a synthetic Tweedie dataset with known base parameters (μ_{true} , p_{true} , ϕ_{true}). Run your SEDR pipeline against the Naive Variance model.
- **Deliverable:** Track Mean Absolute Parameter Error ($|p_{\text{true}} - p_{\text{pred}}|$). Show that SEDR recovers true structural properties faster than alternative setups.

Consolidated Progress Tracking Suite

Run all 4 competitive pipelines through this evaluation matrix at the conclusion of your loop:

Metric	Baseline 1: Random (NSL)	Baseline 2: Naive Variance	Baseline 3: Pure Tree Disagreement	Method 4: Checkpoint Variance + SEDR
Final Tweedie Deviance <i>(Lower is</i>	Baseline Metric	Expect High Error	Expect Moderate Improvement	Target Lowest Out-Of-Sampl e Deviance

<i>Better)</i>				
Final ZMCE <i>(Lower is Better)</i>	Stable Base Line	Expect Massive Spike (Collapse)	Expect Moderate Inflation	Target Lowest Calibration Error
Gini Coefficient <i>(Higher is Better)</i>	Standard Baseline	Poor Performance	Moderate Separation	Target Highest Discrimination Utility